Programmazione in Python
strutture dati: cicli e iteratori

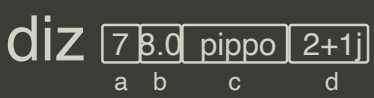
Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

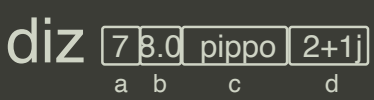
Dizionari e visita dei loro elementi




```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

i = 0
while (i < len(diz)):
    print(diz[i])
    i = i + 1
```

Dizionari e visita dei loro elementi

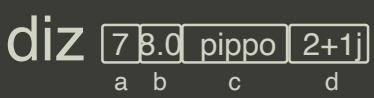



```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

i = 0
while (i < len(diz)):
    print(diz[i])
    i = i + 1
```

```
dario@vulcano: python DizionarioWhileNotWork.py
Traceback (most recent call last):
  File "DizionarioWhileNotWork.py", line 5, in <module>
    print diz[i]
KeyError: 0
dario@vulcano: _
```

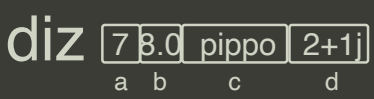
Dizionari e visita dei loro elementi




```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

i = 0
listaChiavi = list(diz.keys())
while (i < len(diz)):
    print(diz[listaChiavi[i]])
    i = i + 1
```

Dizionari e visita dei loro elementi

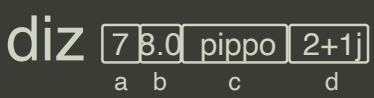



```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

i = 0
listaChiavi = list(diz.keys())
while (i < len(diz)):
    print(diz[listaChiavi[i]])
    i = i + 1
```

```
dario@vulcano: python DizionarioWhile.py
7
pippo
8.0
(2+1j)
dario@vulcano: _
```

Dizionari e visita dei loro elementi

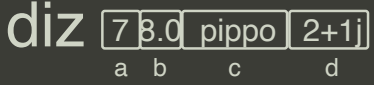



```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

i = 0
listaChiavi = list(diz.keys())
while (i < len(diz)):
    print(diz[listaChiavi[i]])
    i = i + 1
```

!!! esiste un modo più semplice !!!

Iteratori - ciclo for




```
diz = {'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd':
     2 +1j}

for parola in diz:
    print(diz[parola])
```

```
dario@vulcano: python DizionarioFor.py
7
pippo
8.0
(2+1j)
dario@vulcano: _
```

Strutture Controllo - Iterazione: FOR

```
for elemento in iteratore :
    lista comandi
```

Iteratori - ciclo for

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(diz[chiave])
```

```
one
two
three
four
five
```

Iteratori - ciclo for

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(diz[chiave])
```

Cosa succede?

```
one
two
three
four
five
```

Iteratori - ciclo for

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(diz[chiave])
```

```
one
two
three
four
five
```

Cosa succede?

le chiavi del dizionario formano l'insieme iteratore

`diz.keys()` ne restituisce la **vista**

```
>>> diz.keys()
dict_keys(['uno', 'due', 'tre', 'quattro', 'cinque'])
```

Iteratori - ciclo for

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(diz[chiave])
```

```
one
two
three
four
five
```

Cosa succede?

ad ogni iterazione viene estratto un elemento diverso dall'insieme

```
iteratore = uno - valore = one
iteratore = due - valore = two
iteratore = tre - valore = three
iteratore = quattro - valore = four
iteratore = cinque - valore = five
```

Iteratori - ciclo for

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(diz[chiave])
```

```
one
two
three
four
five
```

Cosa succede?

L'insieme iteratore segue l'ordine di inserimento

da Python 3.7; nelle versioni precedenti l'ordine non era garantito

Iteratori - stringhe

```
stringa = 'unaStringa'

for elemento in stringa:
    print(elemento)
```

```
u
n
a
S
t
r
i
n
g
a
```

Iteratori - liste

```
lista = ['uno', 'due', 'tre', 'quattro', 'cinque']

for elemento in lista:
    print(elemento)
```

```
uno
due
tre
quattro
cinque
```

Iteratori - tuple

```
tupla = (1, 2, 'tre', 'quattro', 5)

for elemento in tupla:
    print(elemento)
```

```
1
2
tre
quattro
5
```

Iteratori - dizionari

```
diz = {'uno': 'one', 'due': 'two', 'tre': 'three', 'quattro': 'four', 'cinque': 'five'}

for chiave in diz:
    print(chiave)
```

```
uno
due
tre
quattro
cinque
```

Iteratori - dizionari

```
diz = { 'a': 7, 'b': 3.0 + 5, 'c': 'pippo', 'd': 2 +1j }
for (key, value) in diz.items():
    print(f"chiave: {key} | valore: {value} ||oppure valore: {diz[key]}")
```

restituisce:

```
chiave: a | valore: 7 ||oppure valore: 7
chiave: b | valore: 8.0 ||oppure valore: 8.0
chiave: c | valore: pippo ||oppure valore: pippo
chiave: d | valore: (2+1j) ||oppure valore: (2+1j)
```

Iteratori - range

È possibile costruire un iteratore per eseguire un numero prefissato di iterazioni tramite il comando `range()`.

```
iterabile = range(0, 100, 3)
```

`range(start, stop, step)` costruisce iterabile di interi

- `range()` è la funzione che restituisce un oggetto iterabile di interi, non una lista
- `start` è il valore iniziale, `0` se non si indica
- `stop` è il primo valore **escluso**, l'unico **necessario**: `range(0, 100, 3)` arriva a `99`
- `step` è l'incremento con cui vengono costruiti gli elementi, `1` se non si indica

Iteratori - range

```
lista = ['uno', 'due', 'tre', 'quattro',
     'cinque']
iterabile = range(len(lista))
print(iterabile)

for indice in iterabile:
    print(indice, lista[indice])
```

Iteratori - range

```
lista = ['uno', 'due', 'tre', 'quattro',
     'cinque']
iterabile = range(len(lista))
print(iterabile)

for indice in iterabile:
    print(indice, lista[indice])
```

```
range(0, 5)
0 uno
1 due
2 tre
3 quattro
4 cinque
```

Propietà principali strutture dati

	aggregazione	plasticità	omogeneità	indice	iterabile
scalare	semplice	statica	omogenea		
stringa	complessa	statica	omogenea	intero	si
tupla	complessa	statica	eterogenea	intero	si
lista	complessa	dinamica	eterogenea	intero	si
dizionario	complessa	dinamica	eterogenea	chiave	si - vista